

Documento de Infraestructura - RedVital

Version 2.0 - Sprint 3 / Semana 10

Arkhé Software S.A.S.

7 de octubre de 2026

Control de versiones

Versión	Fecha	Descripción
1.0	22/09/2026	Definición inicial de ambientes, zonas, redes, contenedores, seguridad de borde, observabilidad y despliegue.
2.0	07/10/2026	Alineación con la implementación del Sprint 3: topología de siete VMs, Caddy + APISIX, Kafka 4.3.1, PostgreSQL 16, separación QA/Producción, firewall, secretos, observabilidad, despliegue y verificación extremo a extremo.

1. Propósito

Este documento describe la infraestructura vigente de **RedVital** y cómo se despliegan, comunican, protegen, observan y recuperan sus componentes.

El SAD define las decisiones arquitectónicas. Este documento las materializa mediante:

- máquinas virtuales;
- redes;

- puertos;
- contenedores;
- imágenes;
- secretos;
- firewalls;
- bases de datos;
- Kafka;
- gateway y borde;
- observabilidad;
- despliegue;
- respaldo y restauración;
- verificación operativa.

La infraestructura descrita debe coincidir con la configuración real de los repositorios y ambientes.

2. Alcance

Infraestructura V2 cubre:

1. topología física;
2. distribución de componentes por VM;
3. separación QA / Producción;
4. redes y alcanzabilidad;
5. puertos;
6. Docker y Docker Compose;
7. Caddy;
8. Apache APISIX;
9. Kafka;
10. PostgreSQL;
11. secretos;
12. hardening de contenedores;
13. TLS;
14. firewall;
15. observabilidad;
16. despliegue;
17. rollback;
18. respaldo y restauración;
19. pruebas operativas;
20. riesgos y pendientes.

No sustituye:

- SAD;
- SDD;
- DD;
- ADRs;

- OpenAPI/AsyncAPI;
- plan de pruebas.

3. Relación con otros artefactos

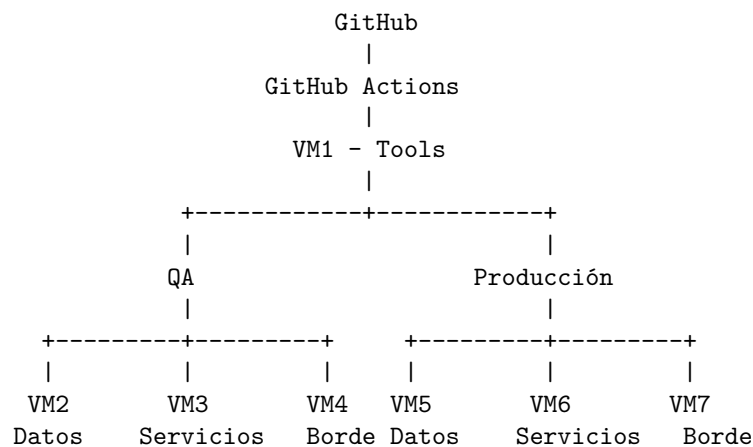
Artefacto	Relación
SAD V3	Define las decisiones estructurales y las fronteras.
SDD V2	Contiene las vistas de diseño y C4, incluida la vista de despliegue.
DD V3	Define propiedad de datos y separación de persistencia.
ADRs	Formalizan Caddy, APISIX, microservicios, Kafka y seguridad.
Repositorio infraestructura-	Fuente de verdad de la composición ejecutable.
Repositorio databases	Define PostgreSQL, roles y migraciones.
Repositorios de servicios	Proveen imágenes/artefactos desplegados.
Testing	Verifica conectividad, seguridad, salud y flujo extremo a extremo.

4. Principios de infraestructura

1. Todos los componentes ejecutables se contenerizan.
2. QA y Producción permanecen separados.
3. Cada base pertenece a un único servicio.
4. Ningún servicio accede a una base ajena.
5. El navegador solo conoce el borde.
6. Caddy es el proxy de borde.
7. APISIX es el API Gateway.
8. La comunicación entre microservicios se realiza por Kafka.
9. Los secretos no se almacenan en Git ni en imágenes.
10. Los contenedores operan con mínimo privilegio.
11. Solo se publican los puertos estrictamente necesarios.
12. Tools no se expone públicamente.
13. La infraestructura debe ser reproducible mediante Docker Compose.
14. Las diferencias entre ambientes se resuelven por configuración.
15. La observabilidad debe estar separada de la ejecución de negocio.
16. Los despliegues rutinarios no dependen de acceso manual por VPN.

5. Topología física vigente

RedVital dispone de **siete máquinas virtuales** en la Nube Javeriana:



Distribución:

VM	Ambiente	Función
VM1	Compartida	Tools, runner, monitoreo y soporte
VM2	QA	Datos
VM3	QA	Servicios + Kafka
VM4	QA	Borde + frontend + APISIX
VM5	Producción	Datos
VM6	Producción	Servicios + Kafka
VM7	Producción	Borde + frontend + APISIX

6. VM1 - Tools

La VM Tools aloja:

- GitHub Actions Runner;
- Prometheus;
- Grafana;
- Kafbat;
- utilidades operativas;
- coordinación de despliegue;
- apoyo a respaldos.

Reglas:

- no publica interfaces administrativas hacia Internet;
- Grafana, Prometheus y Kafbat escuchan en 127.0.0.1;
- el acceso se realiza por túnel SSH;

- una caída de Tools no debe detener servicios ya desplegados;
- su caída sí puede impedir nuevos despliegues y monitoreo centralizado.

7. QA

7.1 VM2 - Datos QA

Aloja PostgreSQL 16.

Bases previstas:

- `db_identidad`;
- `db_campana`;
- `db_donacion`;
- `db_institucional`.

Estado actual:

Base	Estado
<code>db_identidad</code>	esquema implementado
<code>db_campana</code>	esquema implementado
<code>db_donacion</code>	base creada; esquema central pendiente
<code>db_institucional</code>	base creada; esquema central pendiente

Cada base conserva:

- volumen propio;
- credenciales propias;
- red propia;
- migraciones independientes;
- TLS obligatorio;
- respaldo independiente.

7.2 VM3 - Servicios QA

Aloja:

- `identity-service`;
- `campaign-service`;
- `donation-service`;
- Kafka 4.3.1;
- `kafka-init`.

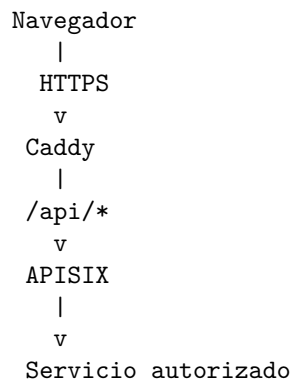
Cuando Institucional y Notificaciones estén implementados, deberán integrarse sin romper la separación de dominios ni introducir acceso cruzado a bases.

7.3 VM4 - Borde QA

Aloja:

- Caddy;
- frontend web;
- Apache APISIX 3.19.

Flujo:



8. Producción

Producción replica la estructura de QA con configuración independiente.

8.1 VM5 - Datos Producción

PostgreSQL 16 con bases independientes por servicio.

8.2 VM6 - Servicios Producción

Microservicios y Kafka.

8.3 VM7 - Borde Producción

Caddy, frontend y APISIX.

Producción usa:

- secretos propios;
- certificados propios;
- claves de firma propias;
- configuración propia;
- datos separados;
- perfiles de Compose sin semillas.

9. Separación QA / Producción

QA y Producción no comparten:

- bases;
- volúmenes;
- credenciales;
- secretos;
- tokens;
- claves de firma;
- archivos `.env`;
- certificados;
- sesiones;
- datos.

La comunicación directa QA -> Producción o Producción -> QA se considera prohibida salvo mecanismo administrativo expresamente autorizado.

10. Docker y Docker Compose

La infraestructura utiliza Docker Engine y Docker Compose.

Estructura de despliegue:

```
infraestructura-/  
  vm-datos/  
  vm-servicios/  
  vm-borde/  
  vm-tools/  
  kafka/  
  ambientes/  
  semillas/qa/  
  scripts/  
  pruebas/
```

Principios:

- composiciones separadas por función;
- imágenes versionables;
- construcción local temporal mientras no exista registro;
- posibilidad de usar `docker save/load` cuando no haya Internet;
- despliegue futuro con imágenes preconstruidas mediante `--no-build`.

11. Caddy

Caddy es el proxy de borde.

Responsabilidades:

- TLS;
- redirección HTTPS;
- HSTS;
- cabeceras de seguridad;
- CSP;
- `X-Frame-Options: DENY`;
- `X-Content-Type-Options: nosniff`;
- `Referrer-Policy: no-referrer`;
- límite de cuerpo de 1 MiB;
- rate limiting;
- entrega de frontend;
- forwarding de `/api/*` hacia APISIX;
- bloqueo de rutas internas.

No contiene reglas de negocio.

12. Apache APISIX

APISIX 3.19 opera detrás de Caddy.

Responsabilidades:

- routing;
- validación inicial de token;
- validación RS256/JWKS;
- control de rol/ruta;
- correlación;
- bloqueo de `/internal`;
- reescritura de rutas;
- denegación de acceso;
- integración con auditoría cuando corresponde.

APISIX no accede a bases de datos de negocio.

13. Kafka

RedVital utiliza **Apache Kafka 4.3.1** en modo KRaft.

Características:

- SASL/SCRAM-SHA-512;
- ACL por servicio;
- `StandardAuthorizer`;
- sin permisos por defecto;
- tópicos creados por `kafka-init`;
- integración asíncrona;
- inspección mediante Kafbat desde Tools.

Principio arquitectónico:

Campañas y Donación no se integran por llamadas HTTP directas;
intercambian eventos mediante Kafka.

Los servicios productores deben usar contratos versionados y, cuando aplique,
Transactional Outbox.

14. PostgreSQL

PostgreSQL 16 se ejecuta en la VM de datos del ambiente correspondiente.

Seguridad:

- `hostssl`;
- TLS 1.2+;
- SCRAM;
- CA por ambiente;
- verificación `verify-ca`;
- redes internas;
- sin publicación indiscriminada;
- roles separados.

Roles:

Rol	Uso
<code>postgres</code>	inicialización local
<code>*_propietario</code>	migraciones
<code>*_servicio</code>	ejecución normal

15. Redes y alcanzabilidad

El principio base es:

Todo tráfico no explícitamente permitido se considera prohibido.

Matriz resumida:

Origen	Destino	Puerto	Propósito
Internet	Borde	80/443	acceso web
Borde	Servicios	8080/8082/8083 según servicio	tráfico API
Servicios	Datos	5432/5433/5434 según composición	persistencia
Servicios	Kafka	9094	eventos

Origen	Destino	Puerto	Propósito
Tools	métricas de VMs	9100/9180/9404 según componente	observabilidad
Administrador	Tools	22	túnel SSH
GitHub/Runner	ambientes	según despliegue	automatización

Los puertos concretos deben seguir la configuración ejecutable del repositorio.

16. Firewall

Los puertos Docker se filtran mediante reglas en `DOCKER-USER`.

El script:

`scripts/firewall.sh`

permite:

- simular;
- instalar;
- reaplicar reglas al reiniciar Docker.

`ufw` no se utiliza como único mecanismo para puertos publicados por Docker.

Reglas:

- bases solo aceptan origen desde Servicios del mismo ambiente;
- servicios solo aceptan tráfico de Borde y componentes explícitamente permitidos;
- Tools solo acepta administración controlada;
- Kafka externo solo desde consumidores/productores autorizados.

17. Secretos

Los secretos se generan fuera del repositorio.

Ubicación:

`secretos/<ambiente>/`

Reglas:

- directorio con permisos restrictivos;
- fuera de Git;
- no usar secretos en imágenes;
- no usar secretos persistidos en código;
- montaje en `/run/secrets`;
- paquetes diferentes por VM;
- la CA privada no se distribuye innecesariamente.

18. Hardening de contenedores

Siempre que la imagen lo permita:

- usuario no root;
- `cap_drop: ALL`;
- `no-new-privileges`;
- filesystem read-only;
- secretos montados;
- puertos mínimos;
- health checks;
- redes mínimas;
- límites de recursos cuando se definan.

19. Orden de arranque

Orden recomendado:

1. VM de Datos;
2. migraciones;
3. VM de Servicios;
4. Kafka y `kafka-init`;
5. servicios;
6. VM de Borde;
7. Tools.

Cada etapa debe finalizar con:

- contenedores `healthy`;
- jobs de inicialización en `Exited (0)`;
- ausencia de errores críticos.

20. Despliegue a QA

Flujo:

```
Commit / PR
  |
  v
GitHub Actions
  |
  v
Runner en Tools
  |
  v
QA
```

Pasos:

1. build o selección de imágenes;
2. validaciones;
3. migraciones;
4. despliegue;
5. health checks;
6. smoke test;
7. verificación E2E;
8. registro del resultado.

QA puede desplegar automáticamente cuando se cumplen los criterios definidos.

21. Despliegue a Producción

Producción requiere:

- pipeline aprobado;
- imagen previamente validada;
- aprobación explícita;
- configuración de Producción;
- migraciones controladas;
- health checks;
- smoke test;
- capacidad de rollback.

No deben usarse semillas de QA.

22. Configuración por ambiente

Archivos:

```
ambientes/qa.env  
ambientes/produccion.env
```

Pueden incluir:

- IPs privadas;
- dominio;
- modo TLS;
- nombres de ambiente;
- referencias no secretas.

Los secretos permanecen fuera de `.env`.

23. TLS

23.1 Borde

QA puede utilizar CA interna de Caddy.

Producción debe utilizar certificado válido para el dominio público, por ejemplo mediante ACME.

23.2 PostgreSQL

TLS obligatorio con CA del ambiente.

23.3 Kafka

Producción utiliza SASL_SSL para el listener externo.

24. Observabilidad

Tools aloja:

- Prometheus;
- Grafana;
- Kafbat.

Se monitorea:

- disponibilidad;
- health;
- CPU;
- RAM;
- disco;
- errores;
- latencia;
- contenedores;
- PostgreSQL;
- Kafka.

Los identificadores de correlación deben permitir unir eventos entre borde, gateway y servicios.

25. Kafbat

Kafbat es herramienta operativa, no componente de negocio.

Acceso:

- únicamente desde Tools;
- mediante túnel SSH;
- con credenciales de inspección;
- sin permisos de productor/consumidor innecesarios.

26. QA y datos sintéticos

QA puede activar:

```
COMPOSE_PROFILES=semillas,metricas
```

Las semillas incluyen cuentas y datos de prueba.

Producción:

```
COMPOSE_PROFILES=metricas
```

sin semillas.

27. Verificación extremo a extremo

El repositorio incluye:

```
pruebas/extremo_a_extremo.py
```

La prueba recorre el sistema desde el borde y ejecuta aproximadamente 45 comprobaciones.

Debe validar, entre otros:

- acceso HTTPS;
- autenticación;
- routing;
- autorización;
- servicios;
- persistencia;
- restricciones de red;
- rate limit;
- eventos cuando aplique.

La prueba no debe acceder directamente a los servicios internos para simular el flujo real del usuario.

28. Validación de firewall

La conectividad debe probarse desde:

1. una máquina permitida;
2. una máquina no permitida.

Ejemplo:

```
nc -zvw3 IP_VM_DATOS 5434
```

La prueba desde un origen no autorizado debe fallar por timeout o rechazo.

29. Respaldo

Los respaldos deben ejecutarse por base.

Contenido mínimo del registro:

- base;
- fecha;
- checksum;
- versión de esquema;
- ambiente;
- ubicación;
- resultado.

Una copia puede mantenerse dentro de la Nube Javeriana para restauración rápida.

Se mantiene como riesgo la ausencia de una copia externa a un dominio de fallo independiente.

30. Restauración

Procedimiento:

1. identificar la base;
2. detener únicamente dependientes;
3. validar respaldo;
4. restaurar;
5. aplicar migraciones posteriores si existen;
6. validar integridad;
7. iniciar servicio;
8. health check;
9. smoke test;
10. registrar tiempos y resultado.

No se requiere restaurar bases no afectadas.

31. Rollback

Cuando una versión debe revertirse:

- desplegar imagen anterior conocida;
- validar compatibilidad de esquema;
- no revertir una migración aplicada editándola;
- ejecutar estrategia explícita de reversión cuando el esquema cambió;
- ejecutar health check;
- smoke test;
- registrar incidente.

32. VPN y administración

La VPN o SSH se utiliza para:

- soporte;
- diagnóstico;
- mantenimiento;
- intervención excepcional.

No constituye la ruta normal de despliegue.

33. Estado actual de infraestructura

Elemento	Estado
Topología 7 VMs	definida
Docker Compose	implementado
Caddy	implementado
APISIX	implementado/configurado
Kafka 4.3.1	implementado
PostgreSQL 16	implementado
Prometheus	implementado/configurado
Grafana	implementado/configurado
Kafbat	implementado/configurado
Firewall scripts	implementados
Secret generation	implementado
QA E2E	script disponible
Registro de imágenes	pendiente
Servicio Institucional	pendiente
Servicio Notificaciones	pendiente
Copia externa de backup	pendiente

34. Riesgos

ID	Riesgo	Impacto	Mitigación
RI-01	VM Tools es punto único de operación	Medio	servicios siguen funcionando; documentar procedimiento manual
RI-02	Kafka es dependencia central de integración	Alto	monitoreo, ACL, persistencia, recuperación y reintentos

ID	Riesgo	Impacto	Mitigación
RI-03	Una sola nube para aplicación y backup	Alto	evaluar segunda copia externa
RI-04	Construcción local de imágenes en VMs	Medio	incorporar registry
RI-05	Institucional/Notificaciones incompletos	Alto	no declarar arquitectura totalmente implementada
RI-06	Error de firewall Docker	Alto	validar DOCKER-USER desde origen permitido/no permitido
RI-07	Divergencia QA/Producción	Medio	mismas imágenes, configuración externa
RI-08	Secretos mal distribuidos	Alto	paquetes específicos por VM y mínimo privilegio

35. Pendientes conocidos

- incorporar un registro de imágenes;
- cerrar implementación de Institucional;
- cerrar implementación de Notificaciones;
- validar topología final sobre las siete VMs reales;
- registrar evidencia de reglas de firewall;
- ejecutar y documentar prueba de restauración;
- definir segunda copia externa de respaldo;
- cerrar métricas reales de CPU/RAM/disco;
- consolidar resultados QA;
- mantener diagramas de despliegue del SDD sincronizados.

36. Regla de mantenimiento

Infraestructura V2 debe actualizarse cuando cambie:

- una VM;
- un puerto;
- una red;

- un servicio desplegado;
- una base;
- una regla de firewall;
- una estrategia de secreto;
- Kafka;
- Caddy;
- APISIX;
- observabilidad;
- backup/restore;
- pipeline;
- ambiente.

37. Criterio de cierre de V2

La versión se considera coherente para Semana 10 cuando:

- las siete VMs coinciden con SAD y SDD;
- QA y Producción están claramente separados;
- Caddy y APISIX aparecen en la capa de borde;
- Kafka aparece en la VM de Servicios;
- las bases están en la VM de Datos;
- Tools contiene Prometheus, Grafana y Kafbat;
- no se documenta comunicación directa entre microservicios;
- no se documenta acceso a bases ajenas;
- Notificaciones no tiene base de negocio;
- los pendientes se distinguen de lo implementado;
- los puertos y seguridad coinciden con la configuración ejecutable.

Anexo A. Mapa resumido por VM

VM	Componentes
VM1 Tools	GitHub Runner, Prometheus, Grafana, Kafbat
VM2 QA Datos	PostgreSQL QA
VM3 QA Servicios	Identidad, Campañas, Donación, Kafka
VM4 QA Borde	Caddy, APISIX, frontend
VM5 Prod Datos	PostgreSQL Producción
VM6 Prod Servicios	microservicios, Kafka
VM7 Prod Borde	Caddy, APISIX, frontend

Anexo B. Flujo permitido

Internet
|

```

      v
Caddy
  |
      v
APISIX
  |
      v
Servicio
  |
  +----> su PostgreSQL
  |
  +----> Kafka ----> consumidor autorizado

Tools ----> métricas / despliegue / soporte

No permitido:

Frontend -----X-----> Servicio directo
Servicio A -----X-----> PostgreSQL de B
Servicio A -----X-----> Servicio B por HTTP directo
Internet -----X-----> APISIX Admin
Internet -----X-----> Prometheus/Grafana/Kafbat

```